

PRATEET TIWARI

prateettiwari29@gmail.com | github.com/Prateet-Github | linkedin.com/in/prateet-tiwari | +91 9569940915

EDUCATION

ABES Engineering College, Ghaziabad, India
Bachelor of Technology in Computer Science and Engineering (CSE)

Expected Aug 2027

EXPERIENCE

Frontend Engineer Intern | [TechSnap](#)
Bengaluru, India (Remote)

Feb 2026 – June 2026

- Developed high-performance dashboards using **Next.js**, improving load time and SEO scores.
- Optimized server-state handling with **TanStack Query**, reducing redundant API calls.
- Implemented scalable state management using **Redux Toolkit**, improving UI predictability.

PROJECTS

[Video Streaming Platform](#) | *Go, Gin, Asynq, AWS S3, Redis, FFmpeg, Next.js, TanStack Query*

- Architected an asynchronous video processing pipeline using **Asynq** and **FFmpeg** workers.
- Reduced backend load by offloading uploads via **AWS S3** pre-signed URLs, enabling direct client-to-S3 uploads.
- Implemented adaptive **HLS streaming** to ensure smooth playback across varying network conditions.

[Distributed Notification System](#) | *TypeScript, NestJS, BullMQ, Redis, Nginx, Firebase, Twilio, Resend*

- Built a distributed notification platform supporting **Email, SMS, Push, and In-App** delivery channels.
- Designed an asynchronous delivery pipeline using **BullMQ** and **Redis**, implementing retries, delivery tracking, user preferences, and worker-based processing.
- Implemented real-time in-app notifications using **SSE** and **Redis Pub/Sub**, integrated **Firebase FCM, Twilio, and Resend**, and added **Nginx** load balancing with CI automation via **GitHub Actions**.

[Secure Payment Microservice](#) | *Node.js, PostgreSQL, Prisma, Stripe, BullMQ, Redis*

- Built a **race-condition safe** payment system using **Stripe** webhooks as the sole source of truth.
- Handled 50+ concurrent requests with zero duplicate charges using **idempotent APIs** and database constraints.
- Load-tested (237 req/s); validated idempotency (12k req/s); handled Stripe rate limits via **retries** and **DLQ**.

[Log Ingestion System](#) | *Fastify, Kafka, Clickhouse, Docker*

- Designed a high-throughput log ingestion system using **Kafka** and **Fastify**.
- Built batch workers for **ClickHouse**, enabling sub-second analytics.
- Containerized all services with **Docker Compose** and automated **CI/CD** pipelines using **GitHub Actions**.

OPEN SOURCE

[Cal.com](#) | Fixed App Store search inconsistencies through input normalization ([PR #29236](#))

[Directus](#) | Fixed incorrect pie chart tooltip values, improving analytics and data accuracy ([PR #27356](#))

TECHNICAL SKILLS

- **Languages:** TypeScript, Go, JavaScript
- **Backend:** Node.js, Express, NestJS, Gin, Socket.io, WebRTC, BullMQ, AsynQ
- **Frontend:** React.js, Next.js, TanStack Query, Redux Toolkit, Tailwind CSS
- **Databases & Storage:** MongoDB, PostgreSQL, Redis, ClickHouse, AWS S3
- **Cloud & DevOps:** Docker, Git & GitHub, GitHub Actions, Vercel, Render